A nightly sealed copy of `~/.claude/projects`

Every Claude Code transcript and memory file, archived and encrypted each night, into iCloud Drive or any folder you choose

The live `~/.claude` stays where it is. Only finished, sealed archives of it travel.

Walkthrough projects archive | Changed 2026-11-12 | Status tested not yet scheduled | For Claude Code on a Mac

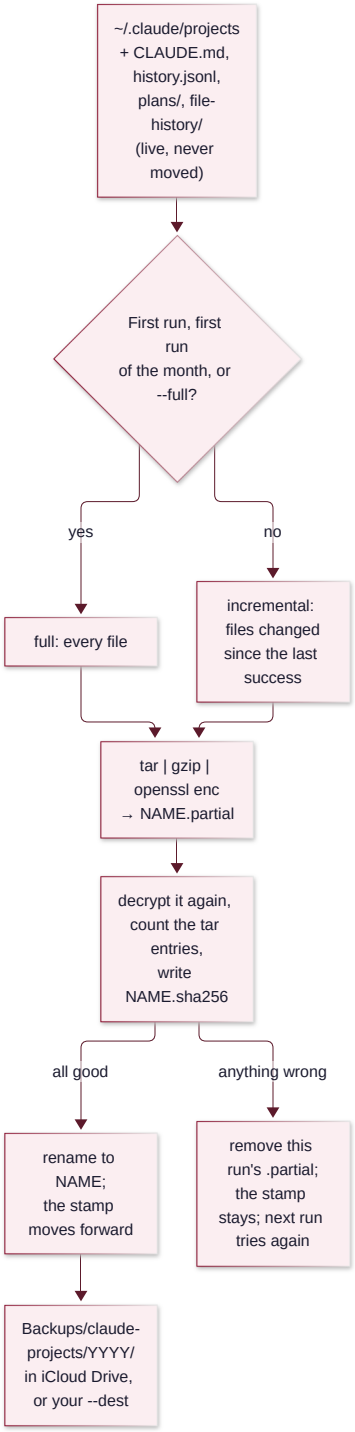
You are here: drasticstatic lab › `my-template` › `walkthroughs/` › `CLAUDE-PROJECTS-ARCHIVE.md` · public · `main` is the working branch

TL;DR

1. **`~/.claude/projects` holds every session you have had,** and your agents' memory. It is not in git and it should never go to GitHub. Until now it lived on one disk. The archive also takes your prompt history (`history.jsonl`), your plans and the snapshots behind checkpoint restore.
2. **One script seals a copy each night.** `tar` , `gzip` , then `openssl` AES-256 with a passphrase that lives in a file on this Mac and in your password manager.
3. **It lands in iCloud Drive by default,** as finished files only. Advanced Data Protection covers it there; the passphrase covers it everywhere else.
4. **A full copy once a month, then only what changed.** Each archive is checked after it is written. Nothing is ever deleted.
5. **Restoring is one line,** into a new, empty folder. Never over the live `~/.claude` .
6. **You set it up with four lines,** below. A helper may not: two of them show something on your screen.
7. **Another folder can be sealed the same way,** in a chain of its own: `--source DIR --name NAME` (below).

> ***Handing this to your AI helper?*** *Say: "Read CLAUDE-PROJECTS-ARCHIVE.md. Run the self-test, tell me what it said, and give me the setup lines. Don't run --init, --run or --install yourself."*

How it works



What goes in	everything under `~/.claude/projects` (transcripts, `memory/` folders, tool results, the manifest and session logs), and, when they exist, `~/.claude/CLAUDE.md`, `~/.claude/history.jsonl`, `~/.claude/plans/` and `~/.claude/file-history/` (below). Nothing inside them is left out: all of it is wanted back.
What stays out	the rest of `~/.claude`: `settings.json`, credentials, caches, plugins, debug logs. Those are not yours to lose in the same way, and some hold tokens.
The script	`scripts/claude-projects-archive.sh`, plain `/bin/sh` and the tools macOS ships with

The proof	scripts/claude-projects-archive.selftest.sh: a fake home in a temporary folder and a stand-in `curl`, 297 checks. Your real `~/.claude` is never read by it, and nothing goes over the network
Names	`claude-projects_20261107-004012_full.tar.gz.enc`, then `..._incr.tar.gz.enc`, each with a `.sha256` beside it. A plain sort of the names is the order they were made, and the order to restore them
The seal	`openssl enc -aes-256-cbc -pbkdf2 -iter 600000 -md sha256 -salt`. The passphrase is read from its file (`-pass file:`), never typed on a command line. LibreSSL (in macOS) and OpenSSL 3 open each other's archives; the self-test checks it
Where the passphrase lives	`~/.config/claude-archive/passphrase`, mode 600, in a folder of mode 700. The script refuses to run if the file is missing or anyone else can read it
The schedule	a LaunchAgent, `com.drasticstatic.claude-projects-archive`, at **00:40** each night, after the trading desk's 23:55 and 23:58 jobs. It runs `--if-due`, so it also runs at login after a night the Mac was off, and does nothing when the last success is under 20 hours old
Logs	`~/Library/Logs/claude-projects-archive/archive.log`, and one row per run in `~/.config/claude-archive/state/<destination>/history.tsv`

What the archive holds, and why the history file matters

Path under `~/.claude/`	What Claude Code keeps there	How long Claude Code keeps it
`projects/`	each session's transcript, its subagents' transcripts and large tool results; each project's auto memory (`memory/`)	transcripts: `cleanupPeriodDays` (30 days by default). Auto memory is not swept
`CLAUDE.md`	your own instructions for every session	until you change it
`history.jsonl`	**every prompt you typed**, with its time and the project's path (it is what the up arrow and `Ctrl+R` recall)	**until you delete it**: the sweep leaves it alone
`plans/`	the plan files written in plan mode	`cleanupPeriodDays`
`file-history/`	the snapshots of files before Claude changed them, behind checkpoint restore	`cleanupPeriodDays`, and only the 100 newest checkpoints

Why `history.jsonl` matters. When the sweep removes a transcript, the conversation is gone from the disk, but the prompts you typed in it are still in `history.jsonl`. For a session swept before this archive ever ran, that file is often the only record left of what you asked, and when. It is plain text, like the transcripts. It is archived whole each night it changes, so it costs a little space every night.

A chain begun before these were added carries on. Nothing needs to be done to an existing destination. The next run is an ordinary incremental: the files of `history.jsonl`, `plans/` and `file-history/` were not in the last run's listing, so they travel in it, however old their dates. A restore from that chain then brings everything back (the self-test proves it on a chain made before and continued after).

> ***Note.*** *If you set `plansDirectory` in your settings, plan mode writes its plans there instead of `~/.claude/plans/` (the path is relative to the project), and this archive does not see them. `claude purge` deletes a project's transcripts, memory, file history and its lines in `history.jsonl`; what an archive already sealed is unaffected.*

Set it up: four lines, in this order

Run each one yourself, in a Terminal or with `!` in a Claude Code session. First prove the script on this Mac (safe for a helper to run; it writes only in a temporary folder):

```
sh ~/code/my-template/scripts/claude-projects-archive.selftest.sh
```

1. Make the passphrase. It writes a random 44-character passphrase into `~/.config/claude-archive/passphrase`, only if there is none yet, and never shows it.

```
! sh ~/code/my-template/scripts/claude-projects-archive.sh --init
```

2. Put the passphrase in 1Password, then clear the clipboard. Make a Password item (for example *claude-projects archive passphrase*) and paste into its password field between these two lines:

```
! pbcopy < ~/.config/claude-archive/passphrase
! pbcopy < /dev/null
```

Without this passphrase nobody can open the archives, you included. If the Mac is lost, the copy in 1Password is the only key.

3. The first archive, by hand. It is a full one: about a minute and 1.2 GB at today's size (measured below).

> ***Warning. This one may put a prompt on your screen.*** *The first time Terminal (or VS Code, or the app you run it from) writes into iCloud Drive, macOS may ask to allow access to files in iCloud Drive. Allow it. iCloud then uploads the archive in the background, which takes a while on a slow connection.*

```
! sh ~/code/my-template/scripts/claude-projects-archive.sh --run
```

4. Schedule it. This writes the LaunchAgent into `~/Library/LaunchAgents/` and loads it.

> ***Note.*** *macOS shows a Background Items Added notification when a LaunchAgent is added. That is expected; the item then appears under System Settings › General › Login Items, where it can also be switched off.*

```
! sh ~/code/my-template/scripts/claude-projects-archive.sh --install
```

Check it.

```
sh ~/code/my-template/scripts/claude-projects-archive.sh --list         # every archive, its size,
the total
launchctl print gui/$(id -u)/com.drasticstatic.claude-projects-archive | grep -E 'state|last exit'
tail ~/Library/Logs/claude-projects-archive/archive.log                 # the morning after
sh ~/code/my-template/scripts/claude-projects-archive.sh --status       # ok, or STALE and why
```

A failed night is never silent. A background job that macOS blocks from iCloud Drive would just stop, and it would look as if you were backed up. So a failed `--run` or `--if-due` is written to `~/.config/claude-archive/state/`, shows a macOS banner (*Claude archive failed*, no prompt), and is passed to `~/.config/claude-archive/notify` if you make that file executable, and sent to your phone if you set up phone notes. `--status` reads it back without touching iCloud: exit 0 when the newest sealed copy is under 36 hours old and nothing failed after it, exit 1 otherwise. On the DEX desk's Mac, the 23:58 journal job asks `--status` every night once the archive is installed and rings the phone when it is stale.

The morning after `--install`, check it once:

```
sh ~/code/my-template/scripts/claude-projects-archive.sh --status   # "ok: ..." or "STALE: ..." and why
grep -i 'not permitted' ~/Library/Logs/claude-projects-archive/archive.log   # nothing printed is good
```

If macOS blocked it (*Operation not permitted* in the log, or `--status` says STALE while the hand run from Terminal worked): the background job may not write into iCloud Drive. Pick one:

1. **Lean: schedule a folder outside iCloud** and keep iCloud for a hand run. `mkdir -p ~/Archives/claude-projects`, then `--install --dest ~/Archives/claude-projects` (Time Machine copies that folder), and once a week `--run` from Terminal for the iCloud copy. `--uninstall` the iCloud schedule if it keeps failing.
2. **Or a drive:** `--install --dest /Volumes/<drive>/claude-projects`; each destination has its own schedule.
3. **Not recommended:** giving `/bin/sh` Full Disk Access in System Settings › Privacy & Security. It would let every shell script read everything.

Turn it off any time: `sh ~/code/my-template/scripts/claude-projects-archive.sh --uninstall`. The plist is moved into `~/.config/claude-archive/retired/`, not deleted, and the archives stay where they are.

Phone notes through ntfy (optional)

ntfy carries a short note from a script to the ntfy app on your phone. With one small file in place, the archive sends **one note when a run fails** and **one when a full archive is sealed** (the monthly one, the first for a new destination, or `--full`). Routine nightly incrementals and "not due" stay quiet, so a note always means something.

What a note says, and what it never says. A failure note carries the Mac's short name, the exit code and the reason, with every word that holds a `/` replaced by `<path>`. A full-archive note carries the archive's own name, its file count and size. No other file name or path leaves the Mac. The topic and token are handed to `curl` on its input, never on a command line, so they do not show in a process list. If ntfy cannot be reached, the run's own result is unchanged and the log gets one line saying so.

1. Make `ntfy.env` in `~/.config/claude-archive/`, which never syncs and is never in a repository. A long random topic name is the only lock on a free ntfy.sh topic, so let the Mac pick one:

```
! mkdir -p -m 700 ~/.config/claude-archive
! ( umask 077; printf 'NTFY_URL=https://ntfy.sh\nNTFY_TOPIC=claude-archive-%s\n' "$(openssl rand -hex 12)" > ~/.config/claude-archive/ntfy.env ) && chmod 600 ~/.config/claude-archive/ntfy.env
```

Already follow a private topic of your own? Put its name after `NTFY_TOPIC=` instead (with any editor; keep the file at mode 600).

2. Subscribe on your phone. Copy the topic name, paste it into the ntfy app (**+**, server `ntfy.sh`), then clear the clipboard:

```
! sed -n 's/^NTFY_TOPIC=//p' ~/.config/claude-archive/ntfy.env | tr -d '\n' | pbcopy
! pbcopy < /dev/null
```

3. Send a test note. It prints only whether it was sent, never the topic or token:

```
! sh ~/code/my-template/scripts/claude-projects-archive.sh --notify-test
```

4. Later, if you reserve the topic or use a server with sign-in, add the access token. Create it on the ntfy web app's *Account* page, copy it there, then run these two lines straight away (the second clears the clipboard). The `grep` must print `1`; if it prints `0`, the clipboard held something else, so fix the line by hand:

```
! ( umask 077; printf 'NTFY_TOKEN=%s\n' "$(pbpaste | tr -d '[:space:]')" >> ~/.config/claude-archive/ntfy.env ) && pbcopy < /dev/null
! grep -c '^NTFY_TOKEN=tk_' ~/.config/claude-archive/ntfy.env
```

Line in `ntfy.env`	
`NTFY_URL=https://ntfy.sh`	optional; `https://` only; a self-hosted server's address goes here
`NTFY_TOPIC=...`	required; letters, digits, `-` and _, up to 64
`NTFY_TOKEN=tk_...`	optional; sent as `Authorization: Bearer`. On ntfy.sh a token opens your whole account, so give it a label and an expiry, keep it out of every repository, and delete it on the Account page if it is ever exposed

The checks. The file is read as data, never run. It is ignored, with one warning in the log, unless it is a plain file you own with mode **600** exactly, and a value with unexpected characters is refused. `--status` says whether phone notes are on. `CLAUDE_ARCHIVE_NO_NTFY=1` turns them off for one run.

Getting it back

The newest state, in one line. It finds the newest full archive and every incremental after it, checks each one's `.sha256`, and unpacks them in order into a folder that must be new or empty:

```
sh ~/code/my-template/scripts/claude-projects-archive.sh --restore-latest ~/claude-restore
```

By hand, any day you choose. Give a full archive, then the incrementals that came after it, oldest first. The script refuses a list that does not start with a full, or is out of order:

```
cd "$HOME/Library/Mobile Documents/com~apple~CloudDocs/Backups/claude-projects/2026"
sh ~/code/my-template/scripts/claude-projects-archive.sh --restore \
  claude-projects_20261201-004012_full.tar.gz.enc \
  claude-projects_20261202-004009_incr.tar.gz.enc \
  claude-projects_20261203-004011_incr.tar.gz.enc \
  ~/claude-restore-dec3
```

Each incremental holds whole files, newer copies of what the full already had plus anything new, so unpacking them in order leaves each file as it was on the last day you gave. Stop at an earlier incremental to see an earlier day.

Check one archive without unpacking it: `--verify FILE` checks the `.sha256`, decrypts to a pipe and reads the tar listing. Nothing is written.

Restore into a folder in your home, as above, not into Desktop or Documents: those may be synced to iCloud Drive, and macOS may ask before a script writes there.

Putting it back. The restore lands in `<folder>/.claude/projects`, `<folder>/.claude/CLAUDE.md` and, where the archives hold them, `history.jsonl`, `plans/` and `file-history/` beside them. Compare first (`diff -rq ~/claude-restore/.claude/projects ~/.claude/projects`). Then quit every Claude Code session and copy back only what you need. The script never writes into `~/.claude`, and refuses to restore there.

On a new Mac. Clone `my-template`, then put the passphrase back from 1Password (copy it in 1Password first; a missing final newline does not matter, tested):

```
mkdir -p -m 700 ~/.config/claude-archive
( umask 077; pbpaste > ~/.config/claude-archive/passphrase ) && pbcopy < /dev/null
sh ~/code/my-template/scripts/claude-projects-archive.sh --restore-latest ~/claude-restore
```

If iCloud has moved older archives off the disk to save space, open the folder in Finder and choose *Download Now* first.

When a transcript is already gone

The archive cannot bring back anything that was deleted before its first run. It seals what is on the disk each night; a transcript the sweep removed earlier was never in any archive. What it does bring back is anything removed *after* it was sealed: restore into a new folder (above) and the transcript is there.

For a session that is gone for good, most of its record usually survives elsewhere: your prompts in `history.jsonl`, the commits it made, its handoff notes, its memory. The tutorial Find, resume or rebuild an old session (its step 7, *When the transcript is gone*) shows how to gather them into a note marked as rebuilt and start a fresh session from it.

Sessions that ran through a gateway

A gateway sits between Claude Code and the model: Claude Code is pointed at it (with `ANTHROPIC_BASE_URL`), and it passes each request on. Some run on the same Mac.

- **The session is an ordinary Claude Code session.** Claude Code's own description of `~/.claude` gives the transcript and the sweep no exception for the way the model is reached (it names one where there is one, as for the session scratchpad). So a session served through a local gateway is written to `~/.claude/projects/` like any other, kept for the same `cleanupPeriodDays`, and sealed by this archive with the rest.
- **The gateway's own log files are a separate thing.** A gateway can log every request it passes on, and each request carries the conversation Claude Code sends the model, so its logs may hold whole conversations. It also holds the provider's key and sees each request's credentials, so its files may hold key material. Their retention is the gateway's own setting, not `cleanupPeriodDays`.
- **Where they belong:** a private local folder, readable only by you. **Never in a repository**, public or private. If you want an off-machine copy, seal that folder with `--source`, not by copying it anywhere as it is. Point the gateway's logs at a folder of their own, so that sealing it takes nothing else with it.

Another destination: no iCloud, or a drive as well

Any folder works: an external drive, a NAS share, another cloud's folder.

```
sh ~/code/my-template/scripts/claude-projects-archive.sh --run --dest /Volumes/Backup/claude-projects
CLAUDE_ARCHIVE_DEST=/Volumes/Backup/claude-projects sh ~/code/my-template/scripts/claude-projects-archive.sh --list
sh ~/code/my-template/scripts/claude-projects-archive.sh --install --dest /Volumes/Backup/claude-projects   # its own nightly job
```

- **Each destination gets its own schedule.** `--install` alone schedules iCloud (`com.drasticstatic.claude-projects-archive`); `--install --dest DIR` adds a second LaunchAgent for that folder (`…archive.dest-<number>` , its own log in `~/Library/Logs/claude-projects-archive/`) beside it, so iCloud and a drive both run nightly. `--uninstall --dest DIR` removes only that one.
- **The folder must already exist.** The script makes only the year folder inside it. That way an unplugged drive does not quietly become a folder on your startup disk; the run is refused instead, and the next night tries again.
- **Each destination keeps its own chain.** The first run into a new destination is a full one.
- **It refuses any destination inside `~/.claude` ,** symlinks followed, and any that contains it.
- Without iCloud Drive and without `--dest` , it refuses and says to pass `--dest` .

Sealing another folder

`--source DIR --name NAME` seals any folder the same way: the same encryption, the same passphrase file, the same check before each archive is renamed, and nothing ever deleted. It works with every mode except `--install` , `--uninstall` , `--init` and `--notify-test` .

```
sh ~/code/my-template/scripts/claude-projects-archive.sh --run --source ~/Notes --name notes
# full, then what changed
sh ~/code/my-template/scripts/claude-projects-archive.sh --list --source ~/Notes --name notes
sh ~/code/my-template/scripts/claude-projects-archive.sh --status --source ~/Notes --name notes
```

- **A chain of its own.** The archives are `notes_20261112-004012_full.tar.gz.enc` , then `..._incr.tar.gz.enc` , in `<destination>/notes/YYYY/` , so they are never taken for the Claude chain's. Their state (the stamp, the listing, the history, a failure) lives in `~/.config/claude-archive/sources/notes/` , so neither chain moves the other's stamp, and a failed `notes` night does not make the Claude chain's `--status` say STALE.
- **Paths are relative to the folder's parent:** an archive of `~/Notes` holds `Notes/...` .
- `NAME` is letters, digits, `-` and `_` , up to 64, beginning with a letter or a digit. All digits (it would look like a year folder) and names beginning `claude-projects` are refused.
- `DIR` **must exist** for `--run` , `--if-due` and `--status` . It may not be inside `~/.claude` (the Claude chain covers what is kept there), hold `~/.claude` or the passphrase, or be, hold or lie inside the destination. Each refusal exits 2 with one line saying why, and writes nothing.

Getting it back. The same as for the Claude chain, with the same two options. `DIR` need not exist any more; the restore goes into a new, empty folder, never into `DIR` itself:

```
sh ~/code/my-template/scripts/claude-projects-archive.sh --restore-latest --source ~/Notes --name notes ~/notes-restore
```

```
diff -rq ~/notes-restore/Notes ~/Notes      # compare before copying anything back
```

Every night, from your own LaunchAgent. `--install` schedules the Claude chain only, for now. For a second chain, make a LaunchAgent of your own (any label, for example `local.archive.notes`) whose program is this one line, at a time of its own:

```
/bin/sh "$HOME/code/my-template/scripts/claude-projects-archive.sh" --if-due --source "$HOME/Notes"
--name notes
```

In the plist, `ProgramArguments` takes it word by word (`/bin/sh`, the script's full path, `--if-due`, `--source`, the folder's full path, `--name`, `notes`; add `--dest` and its folder for a destination other than iCloud). The `post-commit` line below works the same way with `--source` and `--name` added.

Optional: a backup upon commit

If you would like a run soon after you work, rather than at 00:40, add this one line to the `post-commit` hook of a repository you choose (`.git/hooks/post-commit`, or the folder its `core.hooksPath` names, with `#!/bin/sh` as the first line and the file executable):

```
( sh "$HOME/code/my-template/scripts/claude-projects-archive.sh" --if-due >/dev/null 2>&1 & )
```

It runs in the background, so the commit does not wait, and `--if-due` makes it do nothing until 20 hours have passed since the last success. It is opt-in: no fleet hook carries it. In a repository whose hooks are version-controlled (`.githooks/`), keep it out of the shared file: it would then run for everyone who clones it.

What it costs in iCloud space

Measured Nov 6, 2026 on this iMac, into a scratch folder (not iCloud), with a scratch passphrase:

	Files	Before	Sealed	Time
Full archive	1,913	2.8 GB	**1.2 GB**	64 s, plus 7 s to verify it again
One day's incremental (the files changed in the last 24 hours)	285	557 MB	**216 MB**	12 s

Transcripts compress less than plain text would (2.8 GB to 1.2 GB), because they carry pictures and other already-compressed content. An incremental is large because a transcript that grew by one line travels whole.

At that pace: one full a month plus thirty incrementals is roughly **4 to 8 GB a month**, so **50 to 100 GB a year**, and growing as the transcripts grow, because nothing is deleted. That is more than a 50 GB iCloud plan holds. Look at your plan before a year has passed, and use the plan below.

Keeping it smaller, when you decide to. `--prune-plan KEEP` prints what keeping only the newest `KEEP` full archives (and the incrementals after each) would move aside, with the sizes. It moves nothing. Moving them, for example to an external drive, is a person's choice:

```
sh ~/code/my-template/scripts/claude-projects-archive.sh --prune-plan 3          # what would
move; changes nothing
```

```
sh ~/code/my-template/scripts/claude-projects-archive.sh --move-old 3 /Volumes/Backup/claude-projects-older   # move them
```

`--move-old` moves the older chains (never deletes them) into the folder you name, each with its `.sha256`, and checks every one after the move. The newest chains stay in iCloud and still restore on their own. **The rhythm we suggest:** keep three months in iCloud (roughly 15 to 25 GB at today's pace), and once a month move the rest to the backup drive. **Before `--install`, check the plan:** System Settings → your name → iCloud shows the plan and what is used. If iCloud fills, iPhone backups and photo uploads stop too, so leave room.

⚠ Limits, honestly

- **A transcript being written at 00:40 may end mid-line in that night's archive.** The archive takes each file as it is at that moment. The next night's archive has the rest. If a session is running when you restore, read its last line with care.
- **A file deleted from `~/.claude/projects` comes back in a restore** that starts from an older full, because an incremental records what changed, not what was removed. Transcripts are kept for ten years (`cleanupPeriodDays: 3650`), so this rarely matters.
- **A renamed project folder is caught.** `rename-local-repo.py` keeps the files' old modification times, so the time test alone would miss them; each run also keeps the list of every path it saw and adds any path that is new since the last run. The old folder name still comes back in a restore from an older full: archives remember, they do not forget.
- **A run that fails writes nothing and is retried the next night,** or the next time `--if-due` is called. A file that vanishes between the listing and the archive (a project folder moved mid-run, or a checkpoint snapshot that a running session lets go of) fails that night's run; the next run catches up.
- **The first run after `history.jsonl`, `plans/` and `file-history/` joined is larger than usual,** because it carries all of them once. `du -sh ~/.claude/history.jsonl ~/.claude/plans ~/.claude/file-history` shows how much.
- **Whether a LaunchAgent may write into iCloud Drive without asking is UNVERIFIED** on this Mac. A failure is no longer silent (banner, `notify`, `--status`, the desk's nightly check); what to do is under *Check it*: a `--dest` outside iCloud.
- **iCloud takes time to upload.** An archive written just before a disaster may not have left the Mac.
- **The passphrase is the only key.** Lose both copies (this Mac's file and 1Password) and the archives are noise. Advanced Data Protection is a second lock, not a second key.
- **This does not replace Time Machine,** and it does not back up repositories: see iCloud and GitHub, together for those.

📖 Every mode

Command	What it does
`--init`	make `~/.config/claude-archive/` (700) and the passphrase (600) if missing; print the `pbcopy` line, never the passphrase
`--run`	full or incremental, as above; prints the sizes and the destination's total
`--run --full`	a full archive whatever the date
`--if-due`	`--run` only if the last success is over 20 hours old
`--status`	did it work? reads only the state: exit 1 when the newest success is over 36 h old or a failure came after it
`--list`	every archive at the destination, oldest first, with sizes; flags an unfinished `.partial` left by a crash
`--verify FILE`	check the `.sha256`, decrypt to a pipe, read the tar; write nothing
`--restore FULL [INCR ...] DIR`	unpack those, in that order, into a new or empty `DIR`
`--restore-latest DIR`	the newest full and every incremental after it

`--prune-plan KEEP`	print what keeping the newest `KEEP` fulls would move aside; change nothing
`--move-old KEEP DIR`	move (never delete) those older chains into `DIR`, each checked against its `.sha256`
`--install` / `--uninstall`	load or unload the 00:40 LaunchAgent; the plist is moved aside, never deleted. The Claude chain only
`--notify-test`	send one test note through `ntfy.env`; exit 2 if it is missing or unsafe, 1 if the server did not take it
`--dest DIR` (or `CLAUDE_ARCHIVE_DEST`)	another destination, with any mode
`--source DIR --name NAME`	another folder, in a chain of its own, with `--run`, `--if-due`, `--status`, `--list`, `--verify`, `--restore`, `--restore-latest`, `--prune-plan` and `--move-old`

`CLAUDE_ARCHIVE_CONFIG` moves the config folder and `CLAUDE_ARCHIVE_PASSFILE` the passphrase file; the self-test and the measurement above use them so that nothing touches your own.

▼ **Sources for what Claude Code does, checked 2026-11-12**

Claim	Source
What `history.jsonl`, `plans/` and `file-history/` hold; that the sweep deletes `plans/` and `file-history/` after `cleanupPeriodDays` and keeps `history.jsonl` and auto memory; the 100-checkpoint limit; plaintext storage; what `claude purge` deletes; that the scratchpad's conditions name the provider	https://code.claude.com/docs/en/claude-directory
`cleanupPeriodDays` (default 30, minimum 1); `plansDirectory` (default `~/.claude/plans`, relative to the project root)	https://code.claude.com/docs/en/settings-reference
A gateway is reached through `ANTHROPIC_BASE_URL`, holds the provider key, can log every model request, and is passed each request's `Authorization` header	https://code.claude.com/docs/en/llm-gateway

Where a gateway's log files belong, and sealing them with `--source`, are the fleet's practice.

Suggest a change

Noticed something wrong, or a step that could be kinder? Open an issue or a pull request against `main`, changing this page. A person with the role reads it and merges with a merge commit, so your name stays on it. An AI helper may do all of that up to the merge. Not on GitHub? Tell any steward you know, or your helper, in your own words.

Change log

Date	Change
2026-11-12	The archive also takes `history.jsonl`, `plans/` and `file-history/` when they exist; an existing chain carries on with an ordinary incremental. `--source DIR --name NAME` seals another folder in a chain of its own (`<dest>/NAME/YYYY/`, its own state; scheduling it is your own LaunchAgent). New sections: what the archive holds, *When a transcript is already gone*, *Sessions that ran through a gateway*, *Sealing another folder*, and the sources. The self-test grows to 297 checks.
2026-11-10	Optional phone notes through ntfy (`ntfy.env`, mode 600): one on a failed run, one on each full archive, none for incrementals; `--notify-test`; the self-test grows to 184 checks with a stand-in `curl`.
2026-11-06	First version: `claude-projects-archive.sh` and its self-test (104 checks), a real-size measurement into a scratch folder, this page. Not yet run into iCloud, not yet scheduled.

Written 2026-11-06 in the drasticstatic lab. Related: Start Claude in the right folder, iCloud and GitHub, together. Part of the walkthroughs.